If the name of the other model cannot be derived from the association name, you can use the `:class_name` option to supply the model name. For example, if a customer has many orders, but the actual name of the model containing orders is `Transaction`, you'd set things up this way:

```
class Customer < ActiveRecord::Base
  has_many :orders, class_name: "Transaction"
end
```

4.3.2.4 `:counter_cache`

This option can be used to configure a custom named `:counter_cache`. You only need this option when you customized the name of your `:counter_cache` on the belongs_to association.

4.3.2.5 `:dependent`

Controls what happens to the associated objects when their owner is destroyed:

- `:destroy` causes all the associated objects to also be destroyed
- `:delete_all` causes all the associated objects to be deleted directly from the database (so callbacks will not execute)
- `:nullify` causes the foreign keys to be set to `NULL`. Callbacks are not executed.
- `:restrict_with_exception` causes an exception to be raised if there are any associated records
- `:restrict_with_error` causes an error to be added to the owner if there are any associated objects

4.3.2.6 `:foreign_key`

By convention, Rails assumes that the column used to hold the foreign key on the other model is the name of this model with the suffix `_id` added. The `:foreign_key` option lets you set the name of the foreign key directly:

```
class Customer < ActiveRecord::Base
  has_many :orders, foreign_key: "cust_id"
end
```

In any case, Rails will not create foreign key columns for you. You need to explicitly define them as part of your migrations.

4.3.2.7 `:inverse_of`

The `:inverse_of` option specifies the name of the `belongs_to` association that is the inverse of this association. Does not work in combination with the `:through` or `:as` options.

```
class Customer < ActiveRecord::Base
  has_many :orders, inverse_of: :customer
end

class Order < ActiveRecord::Base
  belongs_to :customer, inverse_of: :orders
end
```

4.3.2.8 `:primary_key`

By convention, Rails assumes that the column used to hold the primary key of the association is `id`. You can override this and explicitly specify the primary key with the `:primary_key` option.

Let's say that `users` table has `id` as the primary_key but it also has `guid` column. And the requirement is that `todos` table should hold `guid` column value and not `id` value. This can be achieved like this

```
class User < ActiveRecord::Base
  has_many :todos, primary_key: :guid
end
```

Now if we execute `@user.todos.create` then `@todo` record will have `user_id` value as the `guid`value of `@user`.

4.3.2.9 `:source`

The `:source` option specifies the source association name for a `has_many :through` association. You only need to use this option if the name of the source association cannot be automatically inferred from the association name.